

VOLUMEN II

Roadmap y Sincronización

Plan de evolución del producto y estrategia multi-dispositivo offline-first.

- | | | |
|----|-----------------|--|
| 10 | ROADMAP | Fases 0.1→1.0, versionado, métricas y comunidad |
| 11 | ESTRATEGIA SYNC | Version vectors, conflictos, offline-first, casos especiales |

Tabla de Contenidos

Este volumen compila 2 documentos de VNBOT: Roadmap y Estrategia de Sincronización. El Roadmap define la evolución del producto desde la demo hasta el release estable; la Estrategia de Sync define cómo se sincronizan datos entre múltiples dispositivos con detección y resolución de conflictos.

1. Propósito	5
2. Principios del roadmap	5
3. Estado actual	6
3.1. Completado en documentación	6
3.2. Prototipos visuales existentes	6
3.3. Pendiente de implementación	6
4. Versionado	6
4.1. Semantic Versioning	6
4.2. Canales	6
4.3. Etiquetas de versión	6
5. Fases del roadmap	7
6. Detalle por fase	7
6.1. VNBOT 0.1 — Demo pública en GitHub Pages	7
6.2. VNBOT 0.2 — Núcleo local	7
6.3. VNBOT 0.3 — Servidor privado y Docker	8
6.4. VNBOT 0.4 — Plataformas	8
6.5. VNBOT 0.5 — Memoria grafo	8

6.6. VNBOT 0.6 — MCP Gateway 9

6.7. VNBOT 0.7 — Skills y agentes 9

6.8. VNBOT 0.8 — Integraciones oficiales 9

6.9. VNBOT 0.9 — Estabilización 9

6.10. VNBOT 1.0 — Release estable 10

7. Roadmap funcional por áreas 10

7.1. Memoria 10

7.2. Recordatorios 10

7.3. IA 10

7.4. Visual 10

7.5. MCP 10

8. Matriz de prioridad 10

9. Métricas de release 11

9.1. Producto 11

9.2. Backend 11

9.3. Seguridad 12

9.4. Comunidad 12

10. Dependencias críticas 12

11. Release checklist 12

11.1. Producto 12

11.2. Código 13

11.3. Seguridad 13

11.4. Distribución	13
11.5. Documentación y visual	13
12. Riesgos del roadmap	13
13. Preocupaciones transversales	14
13.1. Testing	14
13.2. Observabilidad	14
13.3. Accesibilidad	14
14. Roadmap comunitario	14
14.1. Contribuciones tempranas	14
14.2. Contribuciones intermedias	15
14.3. Contribuciones avanzadas	15
15. Conclusión	15
1. Propósito	16
2. Principios de sincronización	16
3. Modelo de datos de sincronización	17
3.1. Version vectors	17
3.2. Operación de sync	17
3.3. Detección de conflictos	17
4. Estrategia de resolución	17
4.1. Reglas automáticas (sin intervención del usuario)	17
4.2. Conflictos que requieren al usuario	18
4.3. Flujo de resolución	18

5. Protocolo de sincronización	18
5.1. Push — enviar cambios locales	18
5.2. Pull — recibir cambios del servidor	18
5.3. Resolución de conflicto	18
6. Entidades afectadas por sync	18
7. Casos especiales	19
7.1. Recordatorios durante sync	19
7.2. Borrado de memorias con relaciones	19
7.3. Clock skew	19
8. Implementación por fase	19
9. Riesgos de sincronización	20
10. Seguridad de la sincronización	20
10.1. Conflicto = visibilidad	20
10.2. Reset de sync	21
11. Conclusión	21

DOC 10 • ROADMAP

Plan de evolución del producto: fases 0.1→1.0, versionado, métricas y comunidad.

1. Propósito

Este roadmap define cómo VNBOT evolucionará desde una demostración pública hasta una plataforma open source de memoria personal, recordatorios y mini-agentes. El roadmap no debe interpretarse como una promesa de fechas rígidas. Define prioridades, dependencias y condiciones de salida. Las funciones se incorporan cuando el núcleo anterior es confiable, no simplemente porque sean atractivas o técnicamente posibles.

La prioridad estratégica es:

Confiabilidad
→ privacidad
→ memoria
→ distribución
→ extensibilidad
→ integración

2. Principios del roadmap

Principio	Descripción
Publicar pronto, pero no inseguro	La demo de GitHub Pages puede publicarse temprano porque no procesa datos reales. Las funciones con memoria real, audio, archivos e integraciones deben pasar por controles adicionales.
Cada release utilizable	Una release no debe ser solo una colección de commits. Cada versión debe resolver un flujo concreto y tener documentación, instalación, pruebas, notas de cambios, migraciones y criterios de rollback.
El núcleo no depende de integraciones	Las integraciones externas se añaden como adaptadores. Si Google Calendar, Telegram, Graphify o un proveedor LLM falla, la memoria y los recordatorios internos deben continuar funcionando.
Primero internas, después externas	La creación de una memoria o recordatorio propio es menos riesgosa que enviar un correo o modificar un calendario externo. La autonomía se incrementa por niveles.
Comunidad por módulos	El proyecto debe permitir contribuciones independientes en frontend, UI pixel art, skills, integraciones, documentación, localización, seguridad y modelos/adapters.

3. Estado actual

3.1. Completado en documentación

- PRD, TRD, Esquema Backend, Plan de Implementación.
- Diseño UI/UX, App Flow, Modelo de Datos.
- Seguridad y Privacidad, MCP y Skills.
- Roadmap, Documento Maestro.

3.2. Prototipos visuales existentes

- Golem informático principal y variantes.
- Hoja conceptual de estados de animación.
- Referencias de HUD cyberpunk.
- Dirección visual pixel art definida.

3.3. Pendiente de implementación

- Monorepo funcional.
- Demo navegable.
- Núcleo local.
- Backend real.
- Scheduler y workers.
- APK y desktop.
- MCP Gateway.
- Skills ejecutables.

4. Versionado

4.1. Semantic Versioning

MAJOR.MINOR.PATCH
MAJOR: cambios incompatibles de API, exportación o configuración

4.2. Canales

nightly → builds automáticos para desarrollo
alpha → funciones incompletas para pruebas técnicas

4.3. Etiquetas de versión

v0.1.0-demo
v0.2.0-local
v0.3.0-server
v0.4.0-platforms
v0.4.5-sync
v0.5.0-memory

5. Fases del roadmap

El roadmap se organiza en 11 fases secuenciales, cada una con objetivo, funciones y criterios de salida verificables:

Versión	Nombre	Resultado principal
0.1	Demo pública	GitHub Pages navegable con chat, panel y grafo mock
0.2	Núcleo local	PWA + IndexedDB + memorias y recordatorios offline
0.3	Servidor privado	Docker con API, PostgreSQL/SQLite, workers y scheduler
0.4	Plataformas	APK (Capacitor), Desktop (Tauri), CLI con sync probada
0.5	Memoria grafo	MemoryNode/Edge, procedencia, búsqueda híbrida, contradicciones
0.6	MCP Gateway	Registro, scopes, policy engine, Graphify read-only
0.7	Skills y agentes	7 agentes iniciales, skills versionadas, autonomía limitada
0.8	Integraciones	Calendario, Telegram, Gmail, Outlook, Discord, WhatsApp
0.9	Estabilización	Seguridad, rendimiento, comunidad, plugin SDK
1.0	Release estable	API versionada, backups probados, releases firmados, documentación

6. Detalle por fase

6.1. VNBOT 0.1 — Demo pública en GitHub Pages

Objetivo: presentar visualmente VNBOT sin requerir instalación, cuenta ni proveedor de IA.

Funciones:

- Landing, golem principal, chat simulado.
- Crear recordatorio ficticio, vista Hoy ficticia.
- Grafo de ejemplo, selector de agentes, estados emocionales.
- Presentación de Docker, APK y desktop como próximos canales.
- Documentación enlazada.

Criterios de salida: carga desde GitHub Pages, funciona en móvil y desktop, accesos a documentación y repositorio, navegación principal se entiende en menos de un minuto, tests unitarios con 60% cobertura en core, OpenTelemetry tracing, axe-core sin violaciones AA críticas.

6.2. VNBOT 0.2 — Núcleo local

Objetivo: ofrecer una aplicación personal capaz de funcionar sin servidor remoto.

Funciones: PWA, IndexedDB, bóveda local, chat, heurística, memoria CRUD, grafo básico, recordatorios locales, notificación local, importación/exportación, configuración de zona horaria.

No incluir todavía: MCP remoto, envío de email, WhatsApp, agente autónomo, audio cloud obligatorio.

Criterios de salida: crear memoria sin conexión, crear recordatorio sin LLM, reiniciar y conservar datos, exportar e importar, olvidar memoria y limpiar índices locales.

6.3. VNBOT 0.3 — Servidor privado y Docker

Objetivo: permitir que una persona despliegue VNBOT en un servidor propio.

Componentes: FastAPI, SQLite para instalación sencilla, PostgreSQL para servidor, Redis, worker, scheduler, healthchecks, API /api/v1, autenticación, workspaces, backups.

Distribución:

```
docker compose -f docker-compose.server.yml up -d<br/>vnbot doctor<br/>vnbot migrate
```

Criterios de salida: reiniciar API sin perder datos, reiniciar worker sin perder jobs, ejecutar backup y restore, ver healthchecks, separar usuarios y workspaces, entregar notificaciones internas de forma idempotente.

6.4. VNBOT 0.4 — Plataformas

Objetivo: distribuir VNBOT como APK, desktop y CLI. Requiere sync strategy probada.

APK: Capacitor, permisos de micrófono, notificaciones locales, estado de red, filesystem seguro, APK firmado.

Desktop: Tauri, SQLite local, notificaciones nativas, auto-lock, instaladores para Windows/Linux/macOS.

CLI:

```
vnbot init<br/>vnbot add "revisar presupuesto"<br/>vnbot search "Daniel"<br/>vnbot reminders list<br/>
```

6.5. VNBOT 0.5 — Memoria grafo

Objetivo: convertir la memoria básica en un sistema visible de nodos y relaciones.

Funciones: MemoryNode, MemoryEdge, procedencia, confianza, contradicciones, expiración, búsqueda textual y semántica, recorrido de subgrafos, inspector, vista lista, corrección y olvido, exportación del grafo.

Límites iniciales: profundidad 2-3, Top-K configurable, máximo visible por consulta, sin cargar toda la bóveda en el navegador.

6.6. VNBOT 0.6 — MCP Gateway

Objetivo: conectar herramientas externas de manera segura y visible.

Funciones: registro de servidores, stdio, Streamable HTTP, handshake, descubrimiento, scopes, policy engine, confirmaciones, healthchecks, auditoría, revocación.

Primera integración: Graphify en modo read-only.

Segunda integración: Calendario con lectura y creación de eventos confirmada.

No incluir inicialmente: escritura arbitraria de filesystem, envío automático de emails, acciones financieras, navegación sin sandbox.

6.7. VNBOT 0.7 — Skills y agentes

Objetivo: permitir que el usuario cree asistentes especializados.

Agentes iniciales (7): VNBOT Core, Archivista, Beacon, Navigator, Forge, Sentinel, Scout.

Skills iniciales (11):

`memory.save · memory.search · memory.correct · memory.forget`
`reminder.create · reminder.snooze · r`

6.8. VNBOT 0.8 — Integraciones oficiales

Orden recomendado:

01. Calendario interno.
02. Google Calendar.
03. Telegram Bot API.
04. Gmail lectura/borradores.
05. Outlook.
06. Discord bot oficial.
07. WhatsApp Business Cloud API.

6.9. VNBOT 0.9 — Estabilización

Seguridad: Threat model revisado, pentest básico, SAST/DAST, escaneo de secretos, dependencias auditadas, revisión MCP, revisión de backups, responsable disclosure.

Rendimiento: Pruebas con 10.000 memorias, 1.000 recordatorios, workers, sincronización, grafo, Android gama baja, desktop.

Comunidad: Plugin SDK inicial, documentación de contribución, issue templates, código de conducta, guías de assets, guía de localización.

6.10. VNBOT 1.0 — Release estable

Requisitos:

- API versionada y migraciones documentadas.
- Backups/restores probados.
- Release web, APK, desktop, Docker y CLI estables.
- Seguridad revisada y UI accesible.
- Exportación portable.
- Sistema de skills documentado.
- MCP con scopes y auditoría.
- Guía de administración y guía de privacidad.

VNBOT 1.0 no significa autonomía ilimitada ni todas las integraciones posibles. Significa que el núcleo es confiable, documentado y estable para ampliarse.

7. Roadmap funcional por áreas

Cada área evoluciona en paralelo pero con dependencias claras:

7.1. Memoria

MVP: CRUD y búsqueda
0.5: grafo y procedencia
0.7: scopes por agente
1.0: consolidación, c

7.2. Recordatorios

MVP: puntual/recurrente
0.3: workers y scheduler distribuido
0.4: APK/desktop
0.8: calend

7.3. IA

MVP: heurística
0.3: proveedor local/externo
0.4: router y structured output
0.7: agentes

7.4. Visual

0.1: landing y mascotas
0.2: panel y chat
0.5: grafo
0.7: mascotas por agente
1.0: pac

7.5. MCP

0.6: gateway y tools internas
0.6: Graphify read-only
0.8: integraciones oficiales
1.0: SD

8. Matriz de prioridad

Cada funcionalidad se evalúa por valor, riesgo, dependencia y prioridad resultante:

Funcionalidad	Valor	Riesgo	Prioridad
Fallback heurístico	Alto	Bajo	P0
Testing unitario	Alto	Bajo	P0
Observabilidad (tracing)	Alto	Bajo	P0
Recordatorio puntual	Alto	Bajo	P0
Recurrencia	Alto	Medio	P0
Memoria CRUD	Alto	Medio	P0
Estrategia de sync	Alto	Alto	P1
Accesibilidad WCAG AA	Alto	Medio	P1
Grafo visual	Medio	Bajo	P1
LLM Router	Alto	Medio	P1
Audio local	Alto	Medio	P1
APK	Alto	Medio	P1
Desktop	Medio	Medio	P1
MCP interno	Alto	Alto	P2
Graphify	Medio	Alto	P2
Email send	Medio	Alto	P3
WhatsApp	Medio	Alto	P3
Marketplace skills	Medio	Alto	P4
Multiagente autónomo	Alto	Muy alto	P4

9. Métricas de release

9.1. Producto

- Tiempo hasta primer recordatorio.
- Porcentaje de onboarding completado.
- Tasa de confirmación correcta.
- Memorias recuperadas correctamente.
- Recordatorios completados.

9.2. Backend

- P95 de API.

- Jobs fallidos y jobs reintentados.
- Duplicados.
- Disponibilidad del scheduler.
- Cache hit rate.

9.3. Seguridad

- Vulnerabilidades abiertas.
- Secret scans fallidos.
- Integraciones con scopes excesivos.
- Incidentes.
- Tiempo de respuesta a reportes.

9.4. Comunidad

- Contribuidores.
- PRs.
- Issues.
- Plugins revisados.
- Descargas.
- Instalaciones Docker.

No se deben recopilar contenidos privados para calcular estas métricas. La telemetría es opt-in y agregada.

10. Dependencias críticas

Las fases siguen una cadena de dependencias estricta. No se debe implementar un agente avanzado sobre un sistema de jobs todavía no confiable.

Demo
Frontend base
Modelo de dominio

11. Release checklist

Cada release debe verificar checklist completo en 5 áreas:

11.1. Producto

- [] Objetivo de release documentado.
- [] Funciones fuera de alcance listadas.
- [] Criterios de aceptación verificados.

11.2. Código

- [] Tests, lint, typecheck.
- [] Build, migraciones.

11.3. Seguridad

- [] Secret scan, dependencias.
- [] SAST, permisos, logs.

11.4. Distribución

- [] Artefactos, checksums, SBOM.
- [] Notas, compatibilidad.

11.5. Documentación y visual

- [] README, guía de instalación, variables de entorno.
- [] Guía de upgrade y rollback.
- [] Capturas, assets licenciados, reduced motion, responsive, accesibilidad.

12. Riesgos del roadmap

Riesgo	Consecuencia	Respuesta
Demasiadas integraciones	Mantenimiento inabarcable	Plugins/adapters
Confundir demo con producto	Expectativas incorrectas	Etiquetas claras
MCP sin controles	Exposición de datos	Gateway + policy
Pixel art pesado	Mala experiencia móvil	Sprite sheets/lazy load
LLM costoso	Sostenibilidad baja	Local/fallback/router
Scheduler inestable	Recordatorios duplicados	Locks/idempotencia
Licencias dudosas	Riesgo legal	Inventario/auditoría
Scope creep	Release interminable	P0/P1 congelado
Poca documentación	Comunidad bloqueada	Docs por módulo
Sync sin diseñar	Pérdida de datos multi-dispositivo	Doc 11 antes de 0.3
Testing insuficiente	Regresiones silenciosas	Cobertura mínima por módulo
Accesibilidad postergada	Exclusión de usuarios	WCAG AA desde 0.1
Sin gobernanza clara	Bloqueo de decisiones	Doc 13 en fase 0

13. Preocupaciones transversales

Las siguientes áreas no son fases sino requisitos que aplican a todo el desarrollo:

13.1. Testing

- Unit tests: mínimos por módulo.
- Integration tests: flujos críticos (chat → memoria, recordatorio → notificación).
- E2E tests: recorrido principal por plataforma.
- Contract tests: MCP tools y API endpoints.

13.2. Observabilidad

- OpenTelemetry para tracing distribuido.
- Logs estructurados (JSON).
- Métricas por módulo (latencia P95, error rate, throughput).
- Dashboards por release.

13.3. Accesibilidad

- WCAG 2.2 AA obligatorio.
- axe-core en CI.
- Auditoría manual por release.
- Testing con screen reader antes de v1.0.

14. Roadmap comunitario

14.1. Contribuciones tempranas

- Correcciones de documentación.
- Traducciones.
- Tests.
- Componentes UI.
- Fixtures.
- Accesibilidad.
- Themes.
- Sprites.

14.2. Contribuciones intermedias

- Adapters de storage.
- Proveedores LLM.
- Integraciones oficiales.
- Skills.
- Notificaciones.
- Plugins MCP.

14.3. Contribuciones avanzadas

- Sincronización.
- Graph RAG.
- Workers distribuidos.
- Android nativo.
- Observabilidad.
- Seguridad.

15. Conclusión

VNBOT crecerá en capas. La primera versión no intentará resolver todas las tareas del usuario, sino demostrar una memoria personal y un sistema de recordatorios fiables, privados y extensibles.

La evolución recomendada es:

No se añade autonomía a una base que todavía no puede explicar, auditar y recuperar sus propias acciones.

DOC 11 • ESTRATEGIA DE SINCRONIZACIÓN

Sincronización multi-dispositivo offline-first: version vectors, conflictos y casos especiales.

1. Propósito

Este documento define cómo VNBOT sincroniza datos entre múltiples dispositivos del mismo usuario. La sincronización es un requisito previo a la distribución multiplataforma (APK, Desktop, CLI) y debe estar diseñada y probada antes de que un usuario pueda acceder a sus datos desde más de un cliente.

Sin una estrategia de sync clara, VNBOT se arriesga a: pérdida de datos, duplicados silenciosos, conflictos irresolubles y una experiencia degradada en dispositivos móviles.

2. Principios de sincronización

Principio	Descripción
Offline-first	VNBOT funciona completamente sin conexión. La sincronización es un mecanismo de transporte, no un requisito de operación. Si el servidor está caído, el usuario sigue creando memorias, recordatorios y listas localmente.
Sin pérdida de datos	Nunca se descarta silenciosamente una operación local. Si hay un conflicto, se presenta al usuario. Si hay un error de sync, se reintenta. Si no se puede resolver, se marca para intervención manual.
Conflicto visible, resolución por el usuario	Los conflictos no se resuelven con last-write-wins como política por defecto. El usuario siempre ve qué cambió en cada dispositivo y decide.
Cifrado en tránsito y en reposo	Las operaciones de sync siempre viajan sobre TLS 1.2+. Los datos pendientes de sync en el dispositivo se almacenan cifrados.
Idempotencia	Reenviar la misma operación no debe crear duplicados. Cada operación tiene un ID único por dispositivo.

3. Modelo de datos de sincronización

3.1. Version vectors

Cada dispositivo mantiene un version vector: un mapa de {device_id → counter} que representa el estado de conocimiento del dispositivo. Los version vectors permiten detectar qué operaciones faltan sin necesidad de comparar timestamps.

```
Dispositivo A: {A: 5, B: 3}Dispositivo B: {A: 3, B: 7}  
  
Si A sync con B: A ap
```

3.2. Operación de sync

Cada cambio local genera una sync_operation:

```
{  
  "op_id": "uuid-local-unique",  
  "device_id": "device-A",  
}
```

3.3. Detección de conflictos

Un conflicto ocurre cuando:

- 01. El servidor recibe una operación con base_version que no coincide con la versión actual de la entidad.
- 02. Dos dispositivos envían operaciones para la misma entidad basadas en versiones diferentes.

Ejemplo:

```
Device A: edita memoria v3 → envía op(base: 3)Device B: edita memoria v3 → envía op(base: 3)
```

4. Estrategia de resolución

4.1. Reglas automáticas (sin intervención del usuario)

Tipo de conflicto	Resolución automática	Razón
Mismo campo, mismo valor	Ignorar	No hay diferencia real
Recordatorio completado en A, editado en B	Prevalece completado	La acción de completar es definitiva
Soft delete en A, editado en B	Prevalece delete, ofrece recuperar	El usuario confirmó el borrado
Campos diferentes en la misma entidad	Merge de campos no conflictivos	Cada campo es independiente

Entidad	Sync	Notas
audit_logs	Solo server → cliente	No se sync de cliente a server
settings	Sí	Preferencias del usuario
credentials	No	Se reconfiguran por dispositivo

Las credenciales nunca se sincronizan. Se reconfiguran por dispositivo porque viven en el keychain/keystore del sistema, no en la base de datos.

7. Casos especiales

7.1. Recordatorios durante sync

- Si un recordatorio se crea en el dispositivo A y se sincroniza al servidor, el scheduler del servidor lo toma.
- Si el dispositivo A está offline, el scheduler local del dispositivo dispara el recordatorio.
- Si ambos schedulers disparan el mismo recordatorio, el servidor deduplica por `reminder_id + scheduled_time`.
- Las notificaciones se envían solo desde un punto (server wins si está disponible).

7.2. Borrado de memorias con relaciones

- Al borrar un nodo, sus edges se invalidan.
- Al sincronizar el borrado, el servidor propaga la invalidación.
- Si otro dispositivo creó edges nuevas mientras tanto, se marcan como "huérfanas" y se notifican al usuario.

7.3. Clock skew

- Todos los timestamps se normalizan a UTC en el servidor.
- Si el clock del dispositivo tiene drift > 5 minutos, se muestra una advertencia y se sugiere sincronizar la hora del dispositivo.
- Las operaciones se ordenan por version vector, no por timestamp.

8. Implementación por fase

La sync se introduce progresivamente, no de golpe:

Fase	Sync
0.1 MVP	No. Solo local. Los campos version y updated_at se preparan.
0.2 Servidor	Sync server ↔ browser (misma máquina o red local).
0.3 Plataformas	Sync multi-dispositivo completo con conflict resolution.

La sync multi-dispositivo completa solo se activa en 0.3, después de que la sync server ↔ browser haya sido probada en 0.2. No se salta fases.

9. Riesgos de sincronización

Riesgo	Impacto	Mitigación
Conflictos frecuentes	Experiencia degradada	Minimizar con detección temprana y merge automático de campos
Queue local crece sin límite	Storage agotado	Límite de 10.000 ops pendientes. Alerta al usuario.
Sync parcial falla	Estado inconsistente	Transacciones por batch. Rollback automático.
Device perdido con ops pendientes	Datos perdidos	Las ops se descargan al server en el próximo sync de cualquier dispositivo. Si solo había un dispositivo, el backup local las contiene.
Performance con muchos dispositivos	Latencia alta	Sync es peer-to-server, no peer-to-peer. Un usuario típico tiene 2-3 dispositivos.

10. Seguridad de la sincronización

La sync introduce riesgos específicos que se manejan con controles adicionales:

- La sync nunca envía contenido en texto claro sobre HTTP. Siempre TLS 1.2+.
- El version vector no contiene datos del usuario, solo metadatos operativos.
- Las operaciones pendientes se cifran en reposo (local encryption).
- Un dispositivo no autorizado no puede iniciar sync sin autenticación válida.
- La sync_operations queue se almacena cifrada en el dispositivo.
- Si el dispositivo se pierde o es robado, el cifrado local protege las ops pendientes.
- Las credenciales de autenticación se almacenan en keychain/keystore del sistema.

10.1. Conflicto = visibilidad

Los conflictos de sync nunca se resuelven silenciosamente. Siempre se presentan al usuario con la información suficiente para decidir.

10.2. Reset de sync

La operación sync/full-reset requiere:

- Autenticación completa.
- Confirmación doble con delay de 5 segundos.
- Auditoría inmediata.
- No se permite durante una sync activa.

11. Conclusión

La sincronización multi-dispositivo es el prerequisite técnico para que VNBOT se distribuya en APK, desktop y CLI. Sin una sync confiable, la experiencia multiplataforma se degrada y se arriesgan datos del usuario.

La estrategia se resume en 5 principios:

Offline-first
+ sin pérdida de datos
+ conflicto visible al usuario
+ cifrado en tránsito

La sync no es una feature, es una capa de transporte. El usuario debe poder trabajar sin ella y solo notarla cuando hay conflictos que requieren su decisión.